

Neural Networks and Neural Language Models

Introduction

A modern **neural network** is a network of small computing units, each of which takes a **vector of input** values and produces a **single output value**.

Here, we introduce the neural net applied to classification. The architecture we introduce is called a **feedforward network** because the computation proceeds iteratively from one layer of units to the next.

The use of modern neural nets is often called **deep learning**, because modern networks are often deep (have many layers).

Introduction

Neural networks share much of the same mathematics as **logistic regression**. But neural networks are a more powerful classifier than logistic regression, and indeed a minimal neural network (technically one with a single ‘hidden layer’) can be shown to learn any function.

Neural net classifiers are different from logistic regression in another way. With **logistic regression**, we applied the regression classifier to many different tasks by developing many rich kinds of feature templates based on domain knowledge. When working with **neural networks**, it is more common to avoid most uses of rich hand derived features, instead building neural networks that take **raw words** as inputs and learn to induce features as part of the process of learning to classify.

Introduction

We'll introduce **feedforward networks** as classifiers, and also apply them to the simple task of **language modeling**: assigning probabilities to word sequences and predicting upcoming words.

In subsequent lectures we'll introduce many other aspects of neural models, such as **recurrent neural networks** and **the Transformer**, contextual embeddings like **BERT**, and **encoder-decoder** models and **attention**.

Units

The building block of a neural network is a single computational unit. A unit takes a set of real valued numbers as input, performs some computation on them, and produces an output.

At its heart, a neural unit is taking a weighted sum of its inputs, with one additional term in the sum called a **bias term**.

Given a set of inputs $x_1 \dots x_n$, a unit has a set of corresponding weights $w_1 \dots w_n$ and a bias b , so the weighted sum z can be represented as:

$$z = b + \sum_i w_i x_i$$

$$z = \mathbf{W} \cdot \mathbf{X} + b$$

Activation

Instead of using $\mathbf{z}$, a linear function of $\mathbf{x}$, as the output, neural units apply a **non-linear function f** to $\mathbf{z}$. We will refer to the output of this function as the **activation value** for the unit, $\mathbf{a}$.

Since we are just modeling a single unit, the activation for the node is in fact the final output of the network, which we'll generally call y . So, the value y is defined as:

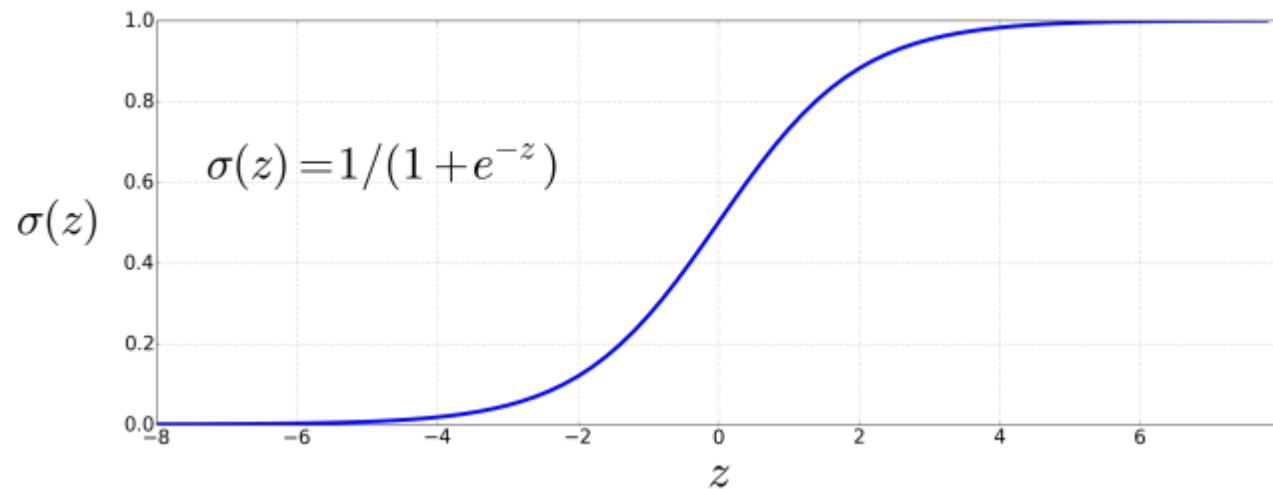
$$y = a = f(z)$$

We'll discuss three popular non-linear functions $f()$ the sigmoid, the tanh, and the rectified linear unit or ReLU.

Sigmoid function

The sigmoid has a number of advantages; it maps the output into the range $[0,1]$, which is useful in squashing outliers toward **0** or **1**. And it's **differentiable**, which will be handy for learning.

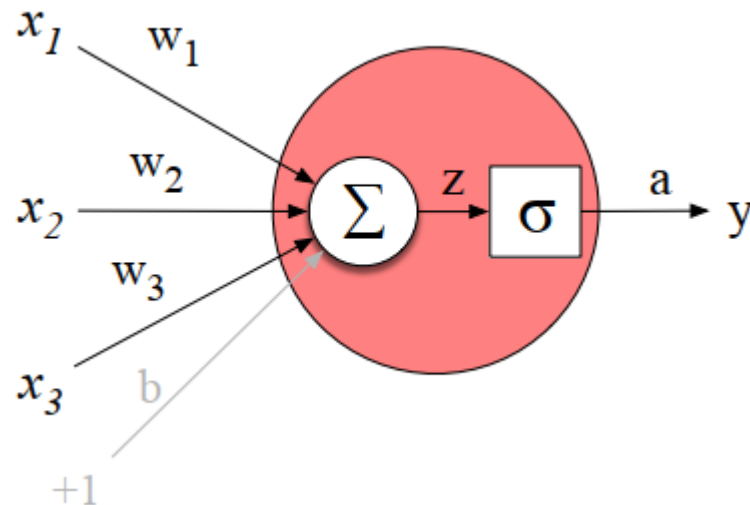
$$y = \sigma(z) = \frac{1}{1+e^{-z}}$$



$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))}$$

Sigmoid function

Fig shows a final schematic of a basic neural unit. In this example the unit takes 3 input values x_1, x_2 and x_3 , and computes a weighted sum, multiplying each value by a weight (w_1, w_2 and w_3 , respectively), adds them to a bias term $\mathbf{b}$, and then passes the resulting sum through a **sigmoid function** to result in a number between 0 and 1.



Sigmoid function

Let's walk through an example just to get an intuition. Let's suppose we have a unit with the following weight vector and bias:

$$\mathbf{w} = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

What would this unit do with the following input vector:

$$\mathbf{x} = [0.5, 0.6, 0.1]$$

The resulting output y would be:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \frac{1}{1 + e^{-(.5*.2 + .6*.3 + .1*.9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

Tanh and ReLU function

In practice, the sigmoid is not commonly used as an activation function. A function that is very similar but almost always better is the **tanh** function shown in below. tanh is a variant of the sigmoid that ranges from -1 to +1:

$$y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

The simplest activation function, and perhaps the most commonly used, is the rectified linear unit, also called the **ReLU**, shown in below. It's just the same as **z** when **z** is positive, and **0** otherwise:

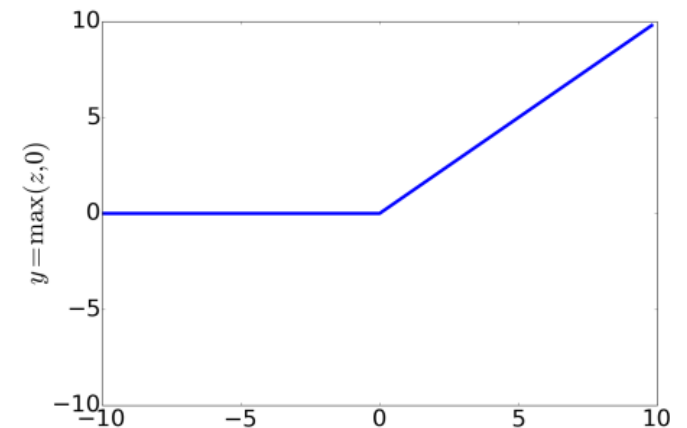
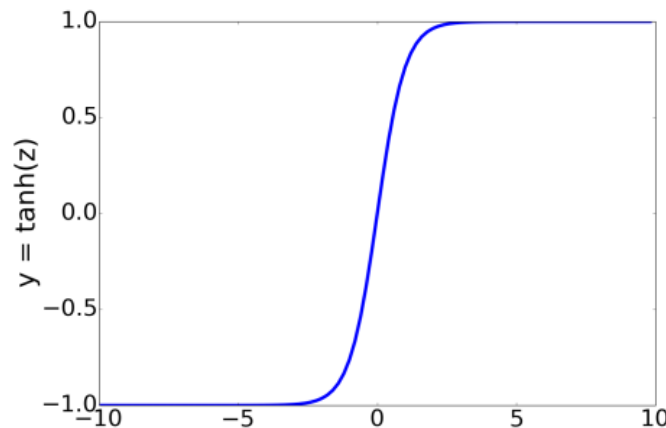
$$y = \text{ReLU}(z) = \max(z, 0)$$

Tanh and ReLU function

These activation functions have different properties that make them useful for different language applications or network architectures.

For example, the **tanh** function has the nice properties of being smoothly differentiable and mapping outlier values toward the mean.

The **rectifier** function, on the other hand has nice properties that result from it being very close to linear.



Activation

In the **sigmoid** or **tanh** functions, very high saturated values of z result in values of y that are saturated, i.e., extremely close to 1, and have derivatives very close to 0.

Zero derivatives cause problems for learning, because as we'll see, we'll train networks by propagating an error signal backwards, multiplying gradients (partial derivatives) from each layer of the network; gradients that are almost 0 cause the error signal to get smaller and smaller until it is too small to be used for training, a problem called the **vanishing gradient problem**.

Rectifiers don't have this problem, since the derivative of ReLU for high values of z is 1 rather than very close to 0.

Perceptron to neural network: XOR

A single neural unit cannot compute some very simple functions of its input. Consider the task of computing elementary logical functions of two inputs, like AND, OR, and XOR.

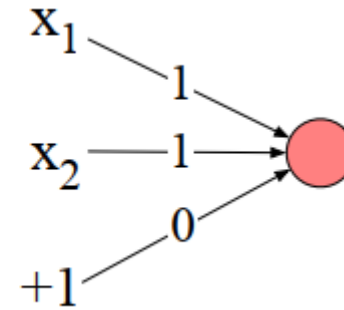
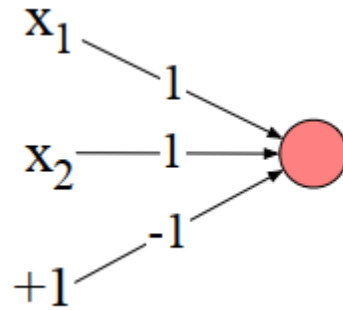
AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

This example was first shown for the perceptron, which is a very simple neural unit that has a binary output and does not have a **non-linear activation** function.

$$y = \begin{cases} 0, & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ 1, & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

Perceptron to neural network: XOR

It's very easy to build a perceptron that can compute the logical AND and OR functions of its binary inputs.



It turns out, however, that it's not possible to build a perceptron to compute logical XOR!

Perceptron to neural network: XOR

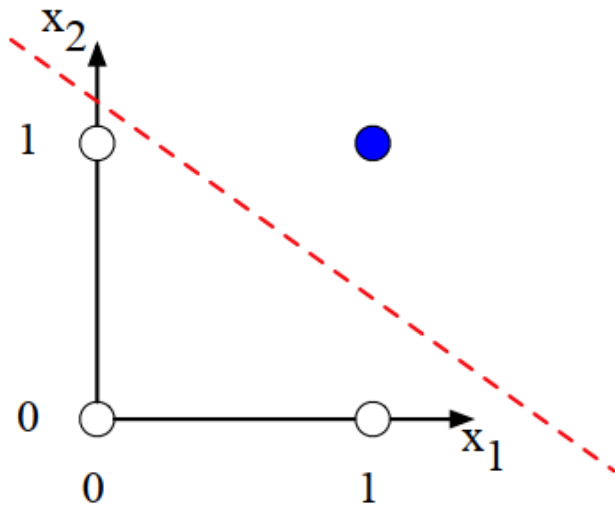
The intuition behind this important result relies on understanding that a perceptron is a **linear classifier**. For a two-dimensional input x_1 and x_2 , the perceptron equation, $w_1x_1 + w_2x_2 + b = 0$ is the equation of a line.

This line acts as a **decision boundary** in two-dimensional space in which the output 0 is assigned to all inputs lying on one side of the line, and the output 1 to all input points lying on the other side of the line.

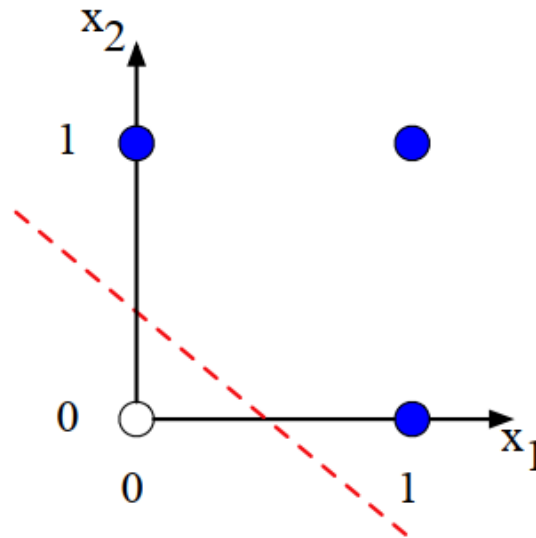
Notice that there is simply no way to draw a line that separates the positive cases of XOR (01 and 10) separable linearly from the negative cases (00 and 11). We say that XOR is not a **linearly separable** function.

Perceptron to neural network: XOR

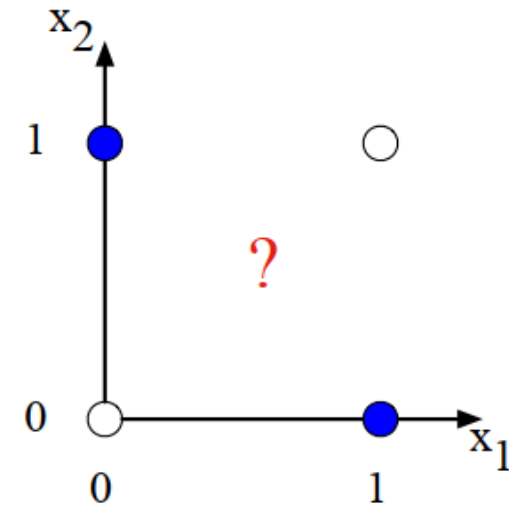
The functions AND, OR, and XOR, represented with input x_1 on the x-axis and input x_2 on the y-axis. Filled circles represent perceptron outputs of 1, and white circles perceptron outputs of 0. There is no way to draw a line that correctly separates the two categories for XOR.



a) x_1 AND x_2



b) x_1 OR x_2

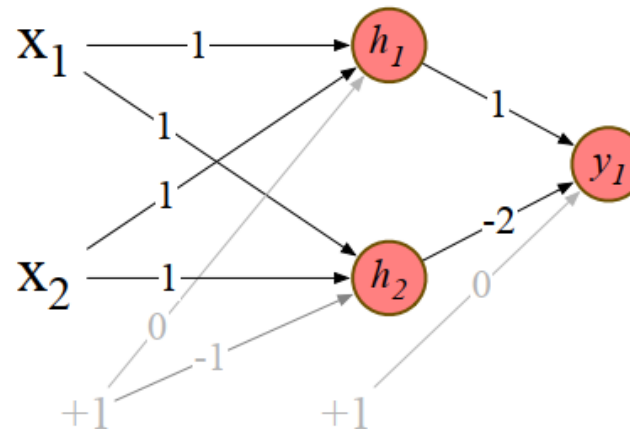


c) x_1 XOR x_2

The solution: neural networks

It can be calculated by a layered network of perceptron units. Rather than see this with networks of simple perceptrons, however, let's see how to compute XOR using two layers of ReLU-based units.

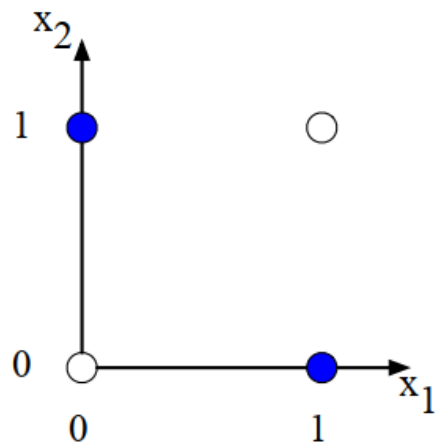
Fig. shows that the input being processed by two layers of neural units. The middle layer (called **h**) has two units, and the output layer (called **y**) has one unit. A set of weights and biases are shown for each **ReLU** that correctly computes the XOR function.



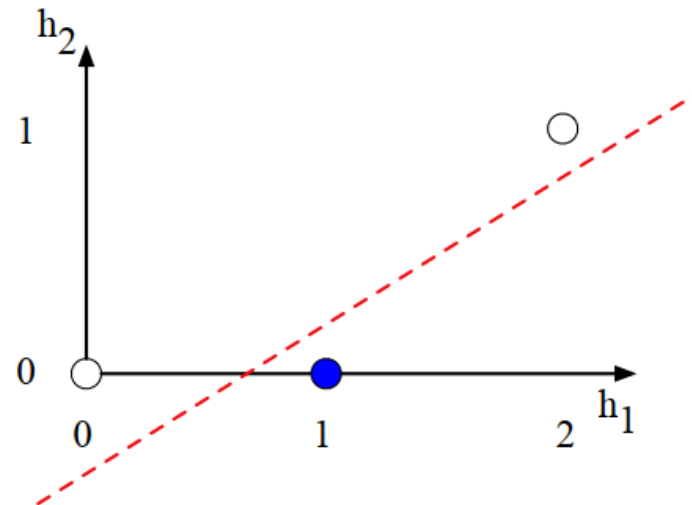
The solution: neural networks

The **hidden layer** forming a new representation of the input. (b) shows the representation of the hidden layer, h , compared to the original input representation x in (a).

Notice that the input point $[0, 1]$ has been collapsed with the input point $[1, 0]$, making it possible to linearly separate the positive and negative cases of XOR.



a) The original x space



b) The new (linearly separable) h space

Feedforward Neural Networks

A feedforward network is a multilayer network in which the units are connected **with no cycles**; the outputs from units in each layer are passed to units in the next higher layer, and no outputs are passed back to lower layers.

Perceptrons are purely linear, but modern networks are made up of units with non-linearities like sigmoids.

Feedforward networks, are sometimes called **multi-layer perceptrons** (or MLPs); this is a technical misnomer.

Simple feedforward networks have three kinds of nodes: input units, hidden units, and output units.

Feedforward Neural Networks

The input layer $\mathbf{x}$ is a vector of simple scalar values.

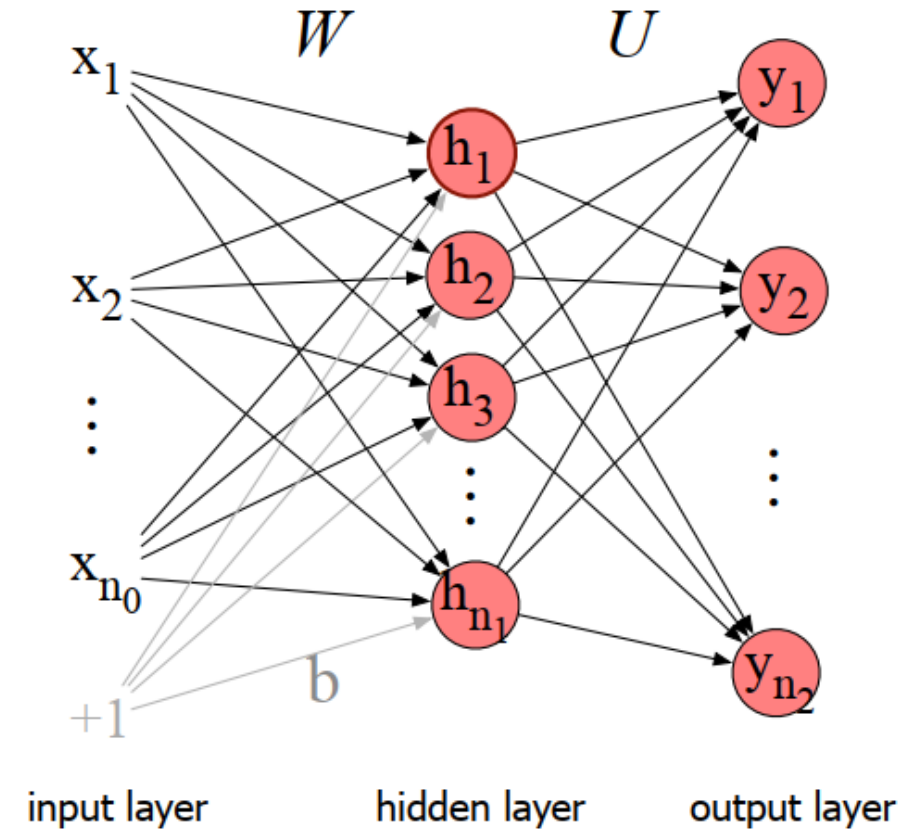
The core of the neural network is the **hidden layer $\mathbf{h}$** formed of hidden units h_i , each of which is a neural unit, taking a weighted sum of its inputs and then applying a **non-linearity**.

In the standard architecture, each layer is **fully-connected**, meaning that each unit in each layer takes as input the outputs from all the units in the previous layer, and there is a link between every pair of units from two adjacent layers. Thus, each hidden unit sums over all the input units.

Feedforward Neural Networks

A simple 2-layer feedforward network, with one hidden layer, one output layer, and one input layer (the input layer is usually not counted when enumerating layers).

Recall that a single hidden unit has as parameters a **weight vector** and a **bias**. We represent the parameters for the entire hidden layer by combining the weight vector and bias for each unit i into a single weight matrix $\mathbf{W}$ and a single bias vector $\mathbf{b}$ for the whole layer.

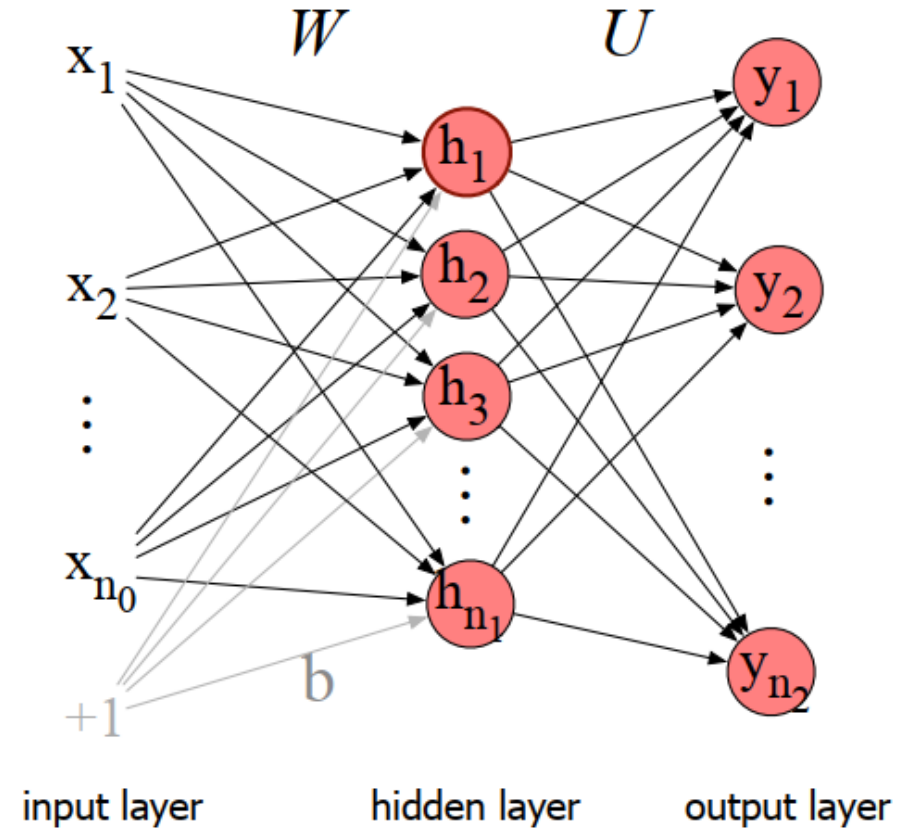


Feedforward Neural Networks

Each element W_{ji} of the weight matrix $\mathbf{W}$ represents the weight of the connection from the i th input unit x_i to the j th hidden unit h_j .

The advantage of using a single matrix $\mathbf{W}$ for the weights of the entire layer is that now the hidden layer computation for a feedforward network can be done very efficiently with simple matrix operations.

In fact, the computation only has three steps: multiplying the weight matrix by the input vector $\mathbf{x}$, adding the bias vector $\mathbf{b}$, and applying the activation function $\mathbf{g}$ (such as the sigmoid, tanh, or ReLU activation function).

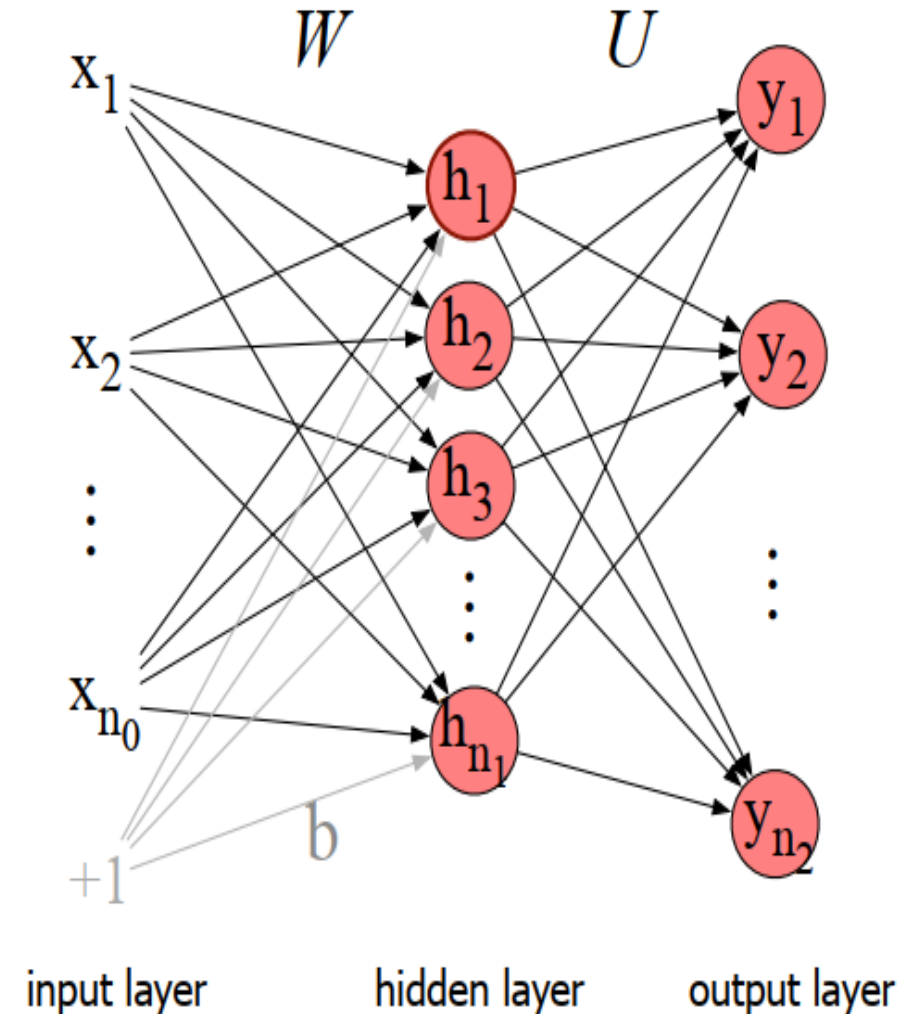


Feedforward Neural Networks

The output of the hidden layer, the vector $\mathbf{h}$, is thus the following (for this example we'll use the sigmoid function σ as our activation function):

$$h = \sigma(Wx + b)$$

Notice that we're applying the σ function here to a vector, while previously it was applied to a scalar.



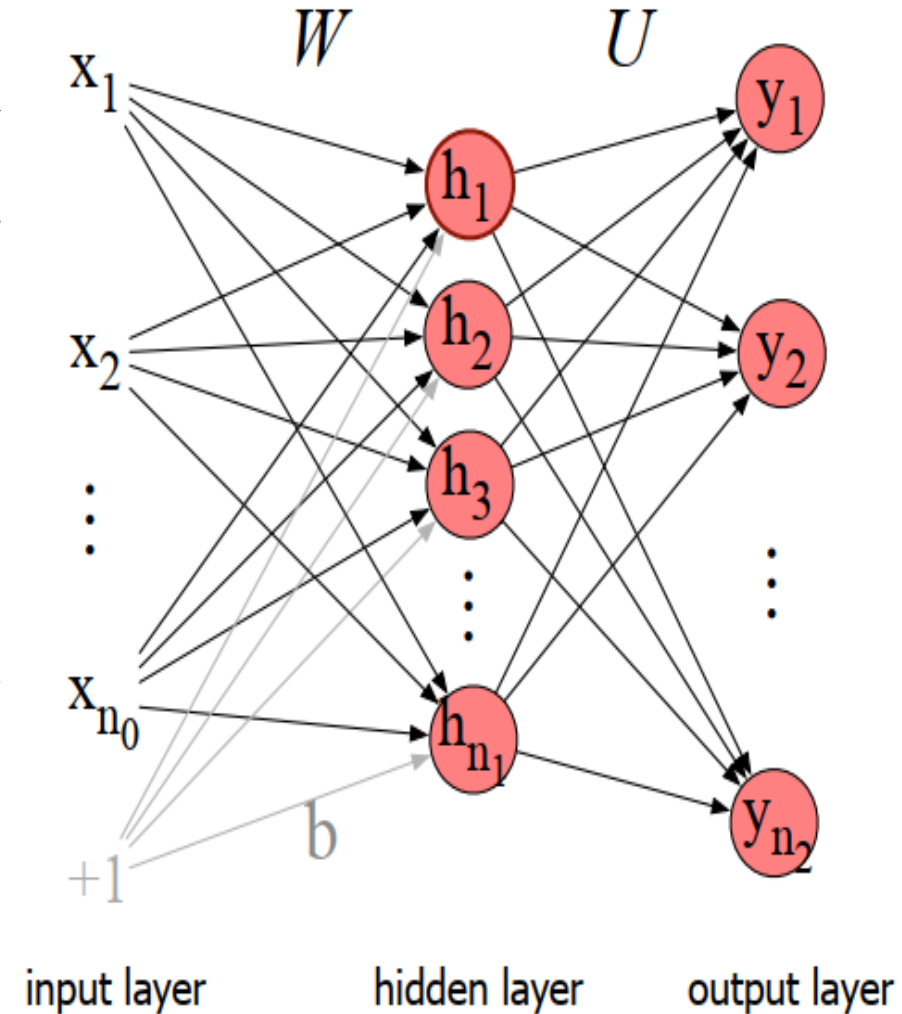
Feedforward Neural Networks

We'll refer to the input layer as layer 0 of the network and have n_0 represent the number of inputs, so $\mathbf{x}$ is a vector of real numbers of dimension n_0 , or more formally $\mathbf{x} \in \mathbb{R}^{n_0}$, a column vector of dimensionality $[n_0, 1]$.

Let's call the hidden layer **layer 1** and the output layer **layer 2**. The hidden layer has dimensionality n_1 , $\mathbf{h} \in \mathbb{R}^{n_1}$ and also $\mathbf{b} \in \mathbb{R}^{n_1}$

And the weight matrix $\mathbf{W}$ has dimensionality

$$\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$$

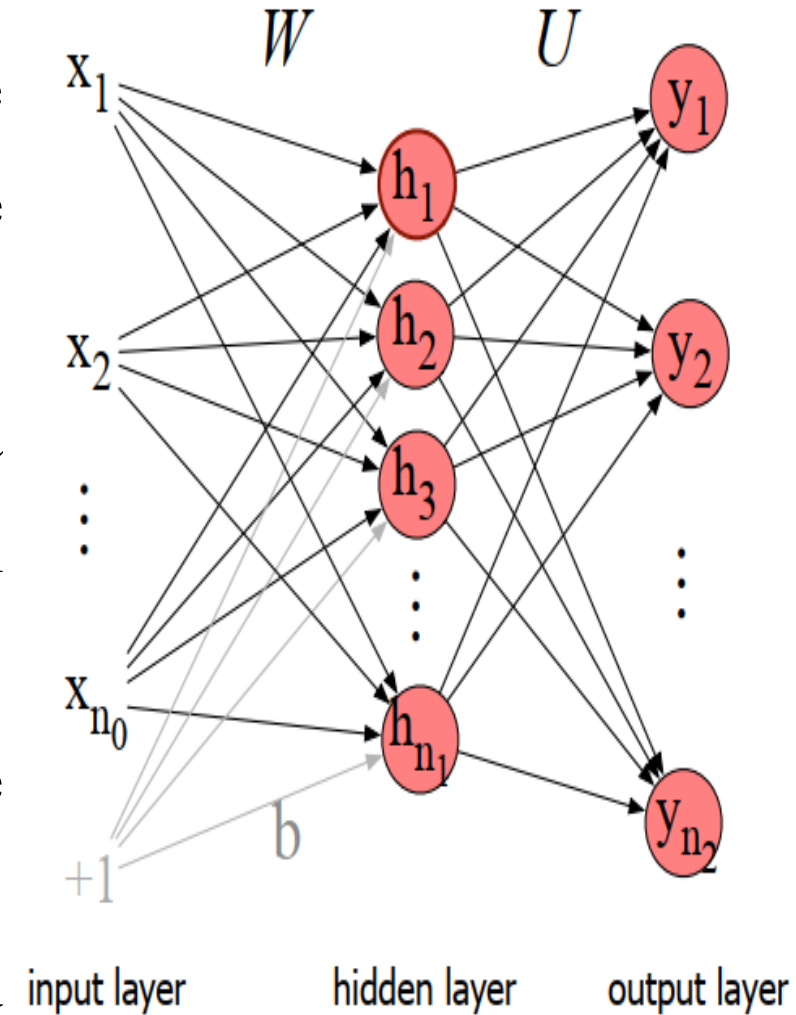


Feedforward Neural Networks

If we are doing a binary task like sentiment classification, we might have a **single output node**, and its scalar value y is the probability of positive versus negative sentiment.

If we are doing **multinomial classification**, such as assigning a part-of-speech tag, we might have one output node for each potential part-of-speech, whose output value is the probability of that part-of-speech, and the values of all the output nodes must sum to one.

The output layer is thus a vector y that gives a probability distribution across the output nodes.

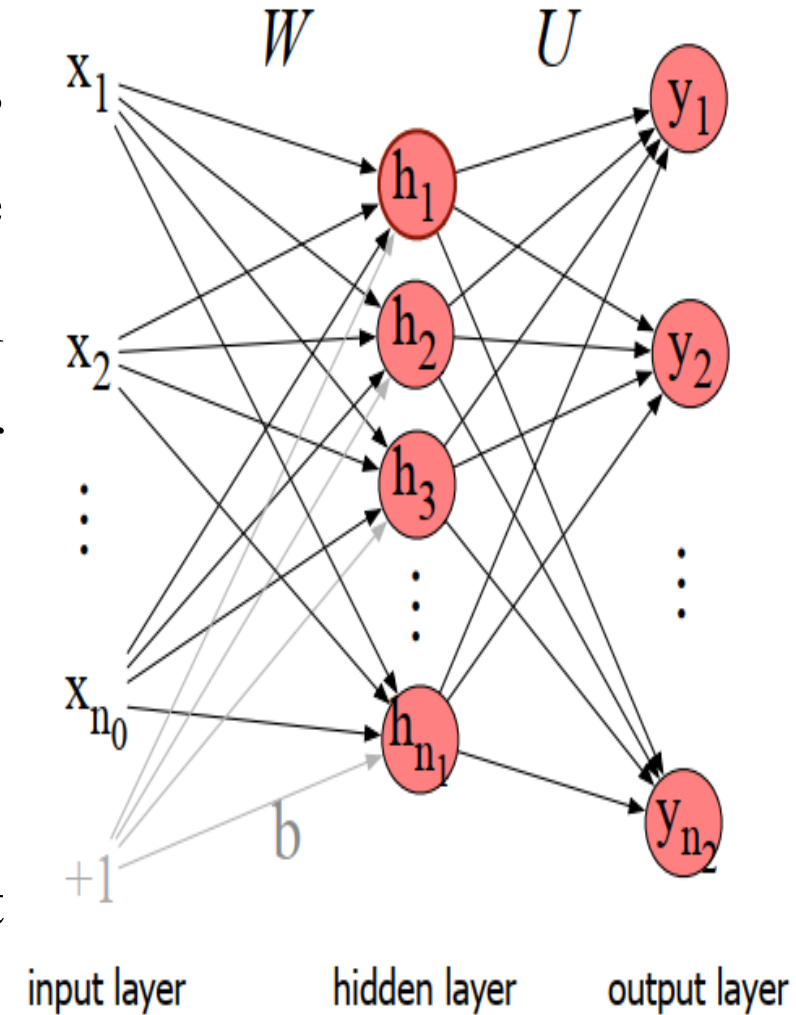


Feedforward Neural Networks

Like the hidden layer, the output layer has a weight matrix (let's call it $\mathbf{U}$), but some models don't include a bias vector $\mathbf{b}$ in the output layer, so we'll simplify by eliminating the bias vector in this example. The weight matrix is multiplied by its input vector ($\mathbf{h}$) to produce the intermediate output $\mathbf{z}$.

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

There are n_2 output nodes, so $\mathbf{z} \in \mathbb{R}^{n_2}$, weight matrix $\mathbf{b} \in \mathbb{R}^{n_1}$ has dimensionality $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$, and element $\mathbf{U}_{ij}$ is the weight from unit $\mathbf{j}$ in the hidden layer to unit $\mathbf{i}$ in the output layer.



Feedforward Neural Networks

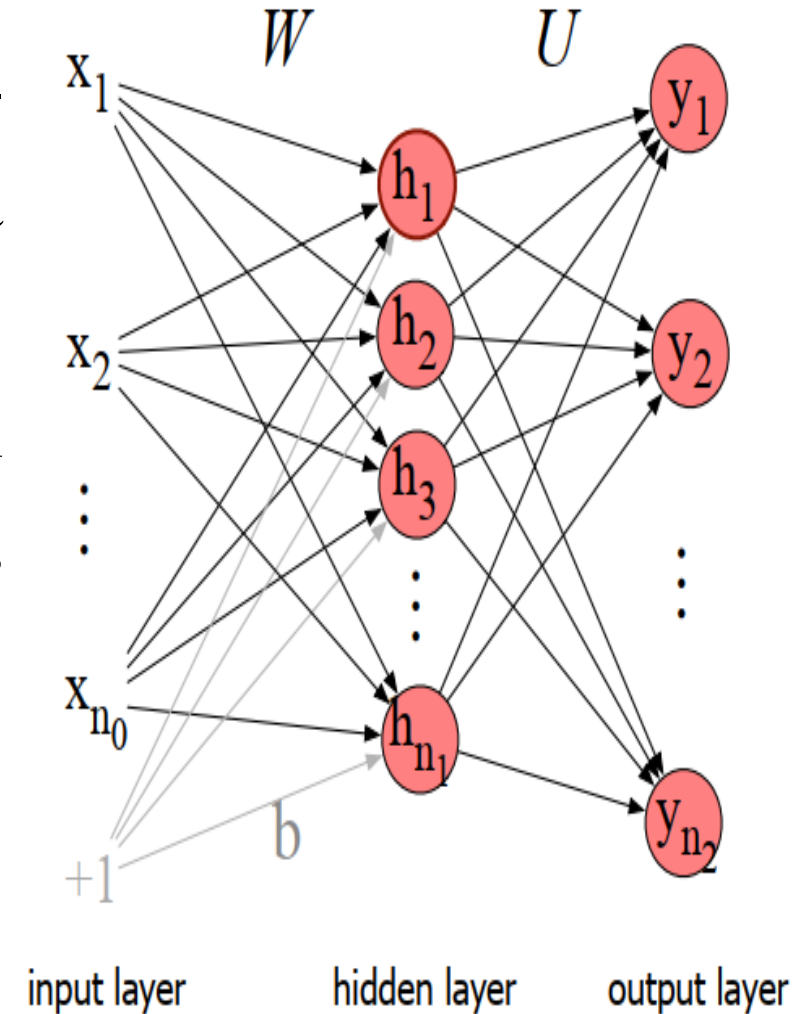
$\mathbf{z}$ can't be the output of the classifier, since it's a vector of real-valued numbers, while what we need for classification is a vector of probabilities.

There is a convenient function for normalizing a vector of real values, by which we mean converting it to a vector that encodes a probability distribution (all the numbers lie between 0 and 1 and sum to 1): the **softmax** function that we saw before.

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d$$

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

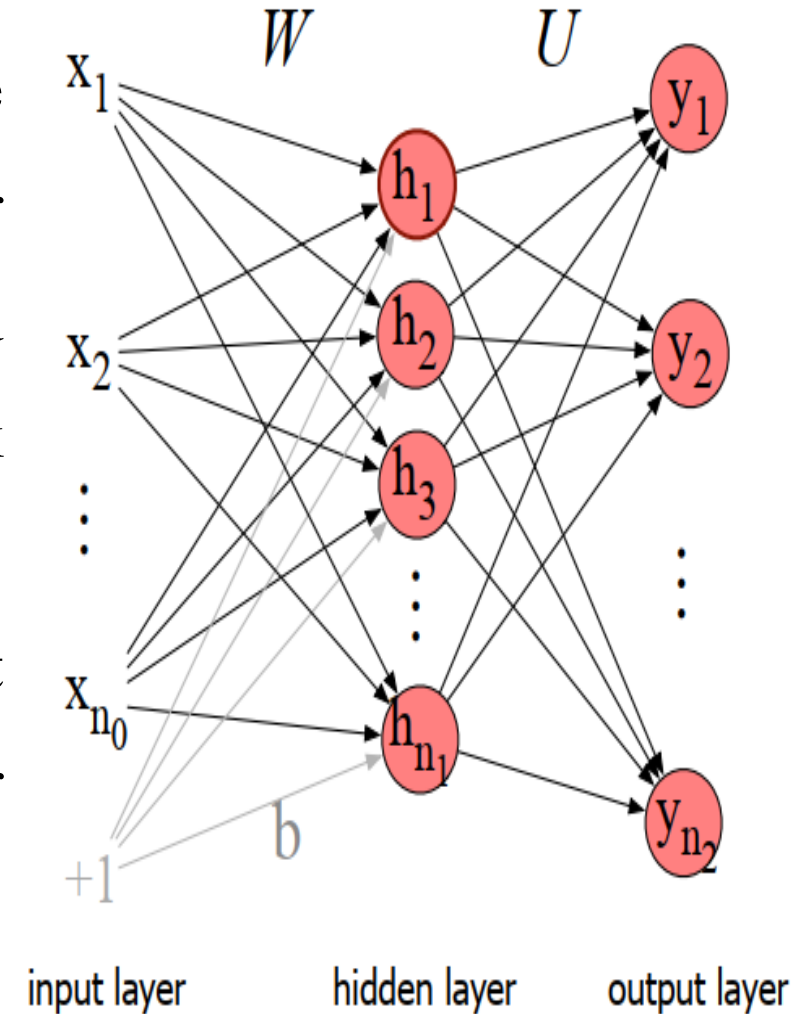
$$\text{softmax}(\mathbf{z}) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010].$$



Feedforward Neural Networks

That means we can think of a neural network classifier with one hidden layer as building a vector $\mathbf{h}$ which is a hidden layer representation of the input, and then running standard multinomial logistic regression on the features that the network develops in $\mathbf{h}$.

So, a neural network is like multinomial logistic regression, but (a) with many layers, since a deep neural network is like layer after layer of logistic regression classifiers;



Feedforward Neural Networks

(b) with those intermediate layers having many possible activation functions (tanh, ReLU, sigmoid) instead of just sigmoid;

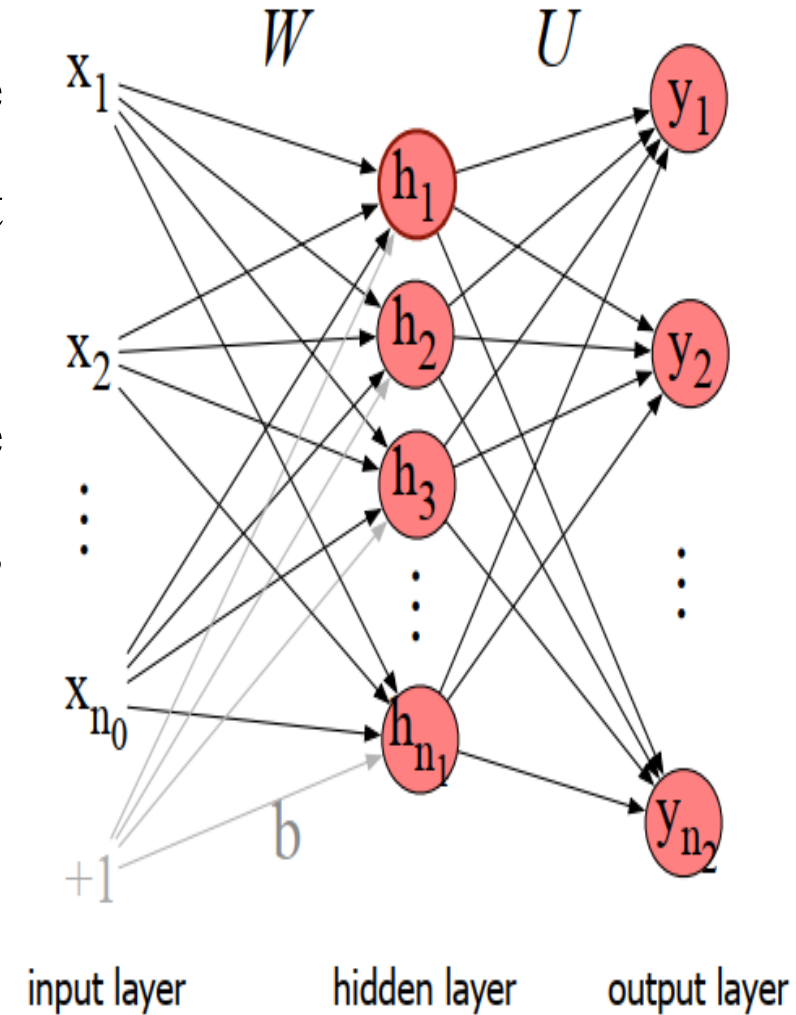
(c) rather than forming the features by feature templates, the prior layers of the network induce the feature representations themselves.

$$h = \sigma(Wx + b),$$

$$z = Uh, y = \text{softmax}(z)$$

$$\mathbf{x} \in \mathbb{R}^{n_0}, \mathbf{h} \in \mathbb{R}^{n_1}, \mathbf{b} \in \mathbb{R}^{n_1}, \mathbf{W} \in \mathbb{R}^{n_1 \times n_0}, \mathbf{U} \in \mathbb{R}^{n_2 \times n_1},$$

$$\mathbf{h} \in \mathbb{R}^{n_2} \text{ and the output } \mathbf{y} \in \mathbb{R}^{n_2}$$



Feedforward networks for NLP: Classification

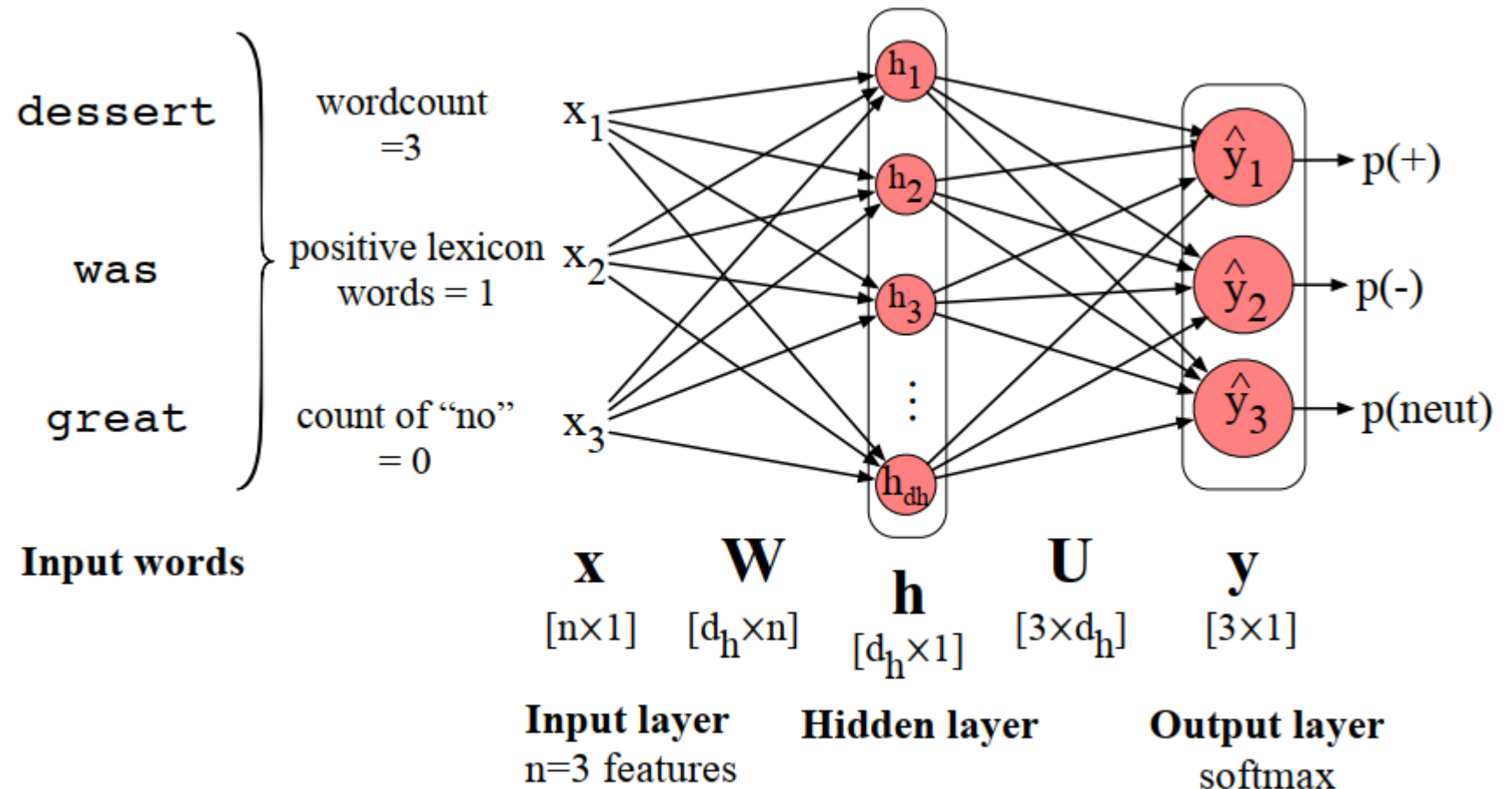
Let's begin with a simple 2-layer sentiment classifier.

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$$

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$



Feedforward networks for NLP: Classification

Most neural NLP applications do something different, however. Instead of using hand-built human-engineered features as the input to our classifier, we draw on deep learning's ability to learn features from the data by representing words as embeddings, like the word2vec or GloVe embeddings.

There are various ways to represent an input for classification. One simple baseline is to apply some sort of pooling function to the embeddings of all the words in the input.

Feedforward networks for NLP: Classification

For example, for a text with n input words/tokens $w_1, w_2, \dots, w_n$, we can turn the n embeddings $e(w_1), \dots, e(w_n)$ (each of dimensionality d) into a single embedding also of dimensionality d by just summing the embeddings, or by taking their mean (summing and then dividing by n):

$$\mathbf{x}_{mean} = \frac{1}{n} \sum_{i=1}^n \mathbf{e}(w_i)$$

$$\mathbf{x} = \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \dots, \mathbf{e}(w_n))$$

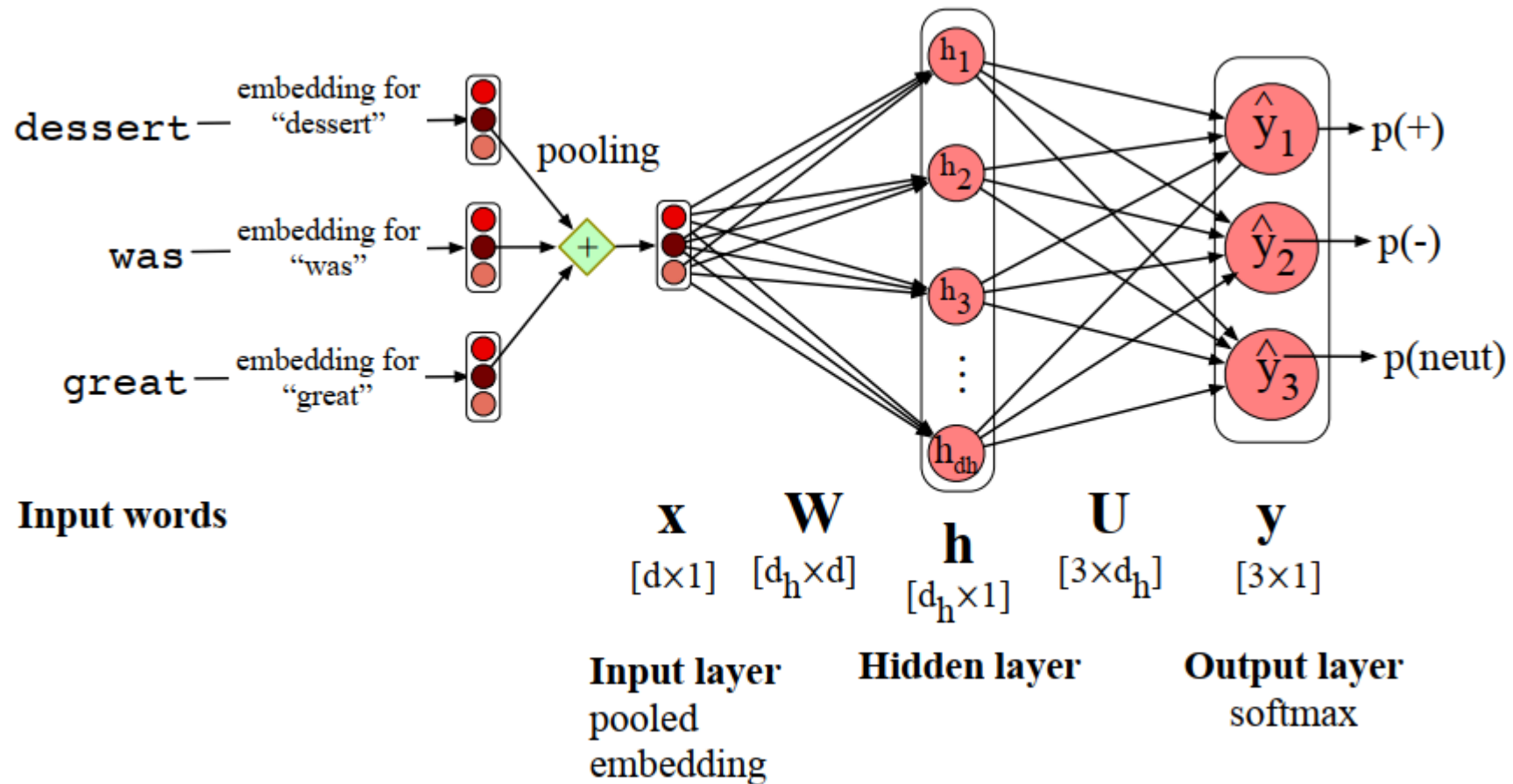
$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

Feedforward networks for NLP: Classification

Feedforward sentiment analysis using a pooled embedding of the input words.



Feedforward Neural Language Modeling

As our second application of feedforward networks, let's consider language modeling: predicting upcoming words from prior word context. Neural language modeling is an important NLP task in itself, and it plays a role in many important algorithms for tasks like machine translation, summarization, speech recognition, grammar correction, and dialogue.

We'll describe simple feedforward neural language models, first introduced by Bengio et al. (2003). While modern neural language models use more powerful architectures like the **recurrent nets** or **transformer networks** to be introduced later, the **feedforward language** model introduces many of the important concepts of neural language modeling.

Feedforward Neural Language Modeling

Neural language models have many advantages over the **n-gram language models** (discussed earlier). Compared to n-gram models, neural language models can handle **much longer histories**, can generalize better over **contexts** of similar words, and are more **accurate** at word-prediction.

On the other hand, neural net language models are much more **complex**, are **slower** and need **more energy** to train, and are less **interpretable** than n-gram models, so for many (especially smaller) tasks an n-gram language model is still the right tool.

Feedforward Neural Language Modeling

A feedforward neural LM is a feedforward network that takes as input at time t a representation of some number of previous words (w_{t-1} , w_{t-2} etc.) and outputs a probability distribution over possible next words.

Thus—like the n-gram LM—the feedforward neural LM approximates the probability of a word given the entire prior context $P(w_t | w_{1:t-1})$ by approximating based on the $N-1$ previous words:

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1})$$

Forward inference in the neural LM

Forward inference is the task, given an input, of running a forward pass on the network to produce a probability distribution over possible outputs, in this case next words.

We first represent each of the N previous words as a one-hot vector of length $|V|$. A one-hot vector is a vector that has one element equal to 1—in the dimension corresponding to that word’s index in the vocabulary— while all the other elements are set to zero. Thus, in a one-hot representation for the word “toothpaste”, supposing it is V_5 , as shown here:

$$\begin{array}{cccccccccccc} [0 & 0 & 0 & 0 & 1 & 0 & 0 & \dots & 0 & 0 & 0 & 0] \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & \dots & \dots & & |V| \end{array}$$

Forward inference in the neural LM

The embedding weight matrix E has a column for each word, each a column vector of d dimensions, and hence has dimensionality $d \times |V|$. Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects out the relevant column vector for word i , resulting in the embedding for word i .

The diagram illustrates the forward inference process in a neural language model. It shows the multiplication of an embedding weight matrix E (dimensions $d \times |V|$) by a one-hot vector (dimensions $|V| \times 1$) to produce the embedding vector e_5 (dimensions $d \times 1$).

The matrix E is represented as a light blue rectangle with a vertical green strip on the left, indicating the selected column. The dimensions d and $|V|$ are labeled. The one-hot vector is a black vertical bar with a white square at the 5th position, labeled 1 at the top and 5 at the bottom. The resulting embedding vector e_5 is a green vertical bar with a 1 at the top and e_5 at the bottom.

Forward inference in the neural LM

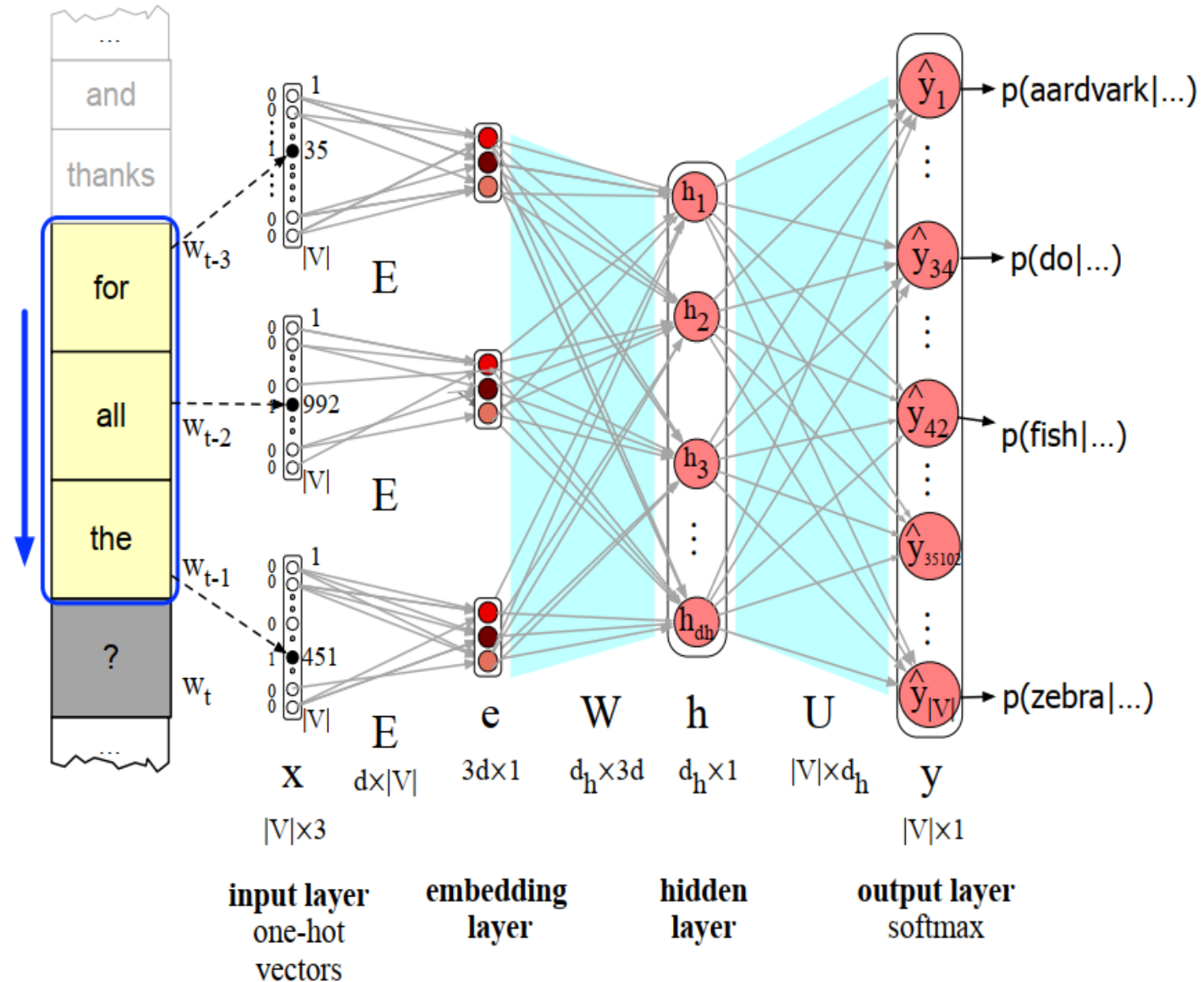
Forward inference in a feedforward neural language model. At each timestep t the network computes a d -dimensional embedding for each context word (by multiplying a one-hot vector by the embedding matrix E), and concatenates the 3 resulting embeddings to get the embedding layer e .

$$\mathbf{e} = [\mathbf{E}\mathbf{x}_{t-3}; \mathbf{E}\mathbf{x}_{t-2}; \mathbf{E}\mathbf{x}_{t-1}]$$

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{e} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$



Training Neural Nets

A feedforward neural net is an instance of supervised machine learning in which we know the correct output $\mathbf{y}$ for each observation $\mathbf{x}$.

What the system produces, is $\hat{\mathbf{y}}$, the system's estimate of the true $\mathbf{y}$. The goal of the training procedure is to learn parameters $\mathbf{W}^{[i]}$ and $\mathbf{b}^{[i]}$ for each layer i that make $\hat{\mathbf{y}}$ for each training observation as close as possible to the true $\mathbf{y}$.

First, we'll need a loss function that models the distance between the system output and the gold output, and it's common to use the loss function used for logistic regression, the **cross-entropy** loss.

Training Neural Nets

Second, to find the parameters that minimize this loss function, we'll use the **gradient descent** optimization algorithm.

Third, gradient descent requires knowing **the gradient** of the loss function, the vector that contains the partial derivative of the loss function with respect to each of the parameters.

In logistic regression, for each observation we could directly compute the derivative of the loss function with respect to an individual $\mathbf{w}$ or $\mathbf{b}$. But for neural networks, with millions of parameters in many layers, it's much harder to see how to compute the partial derivative of some weight in layer 1 when the loss is attached to some much later layer.

Training Neural Nets

How do we partial out the loss over all those intermediate layers?

The answer is the algorithm called **error backpropagation** or **backward differentiation**.

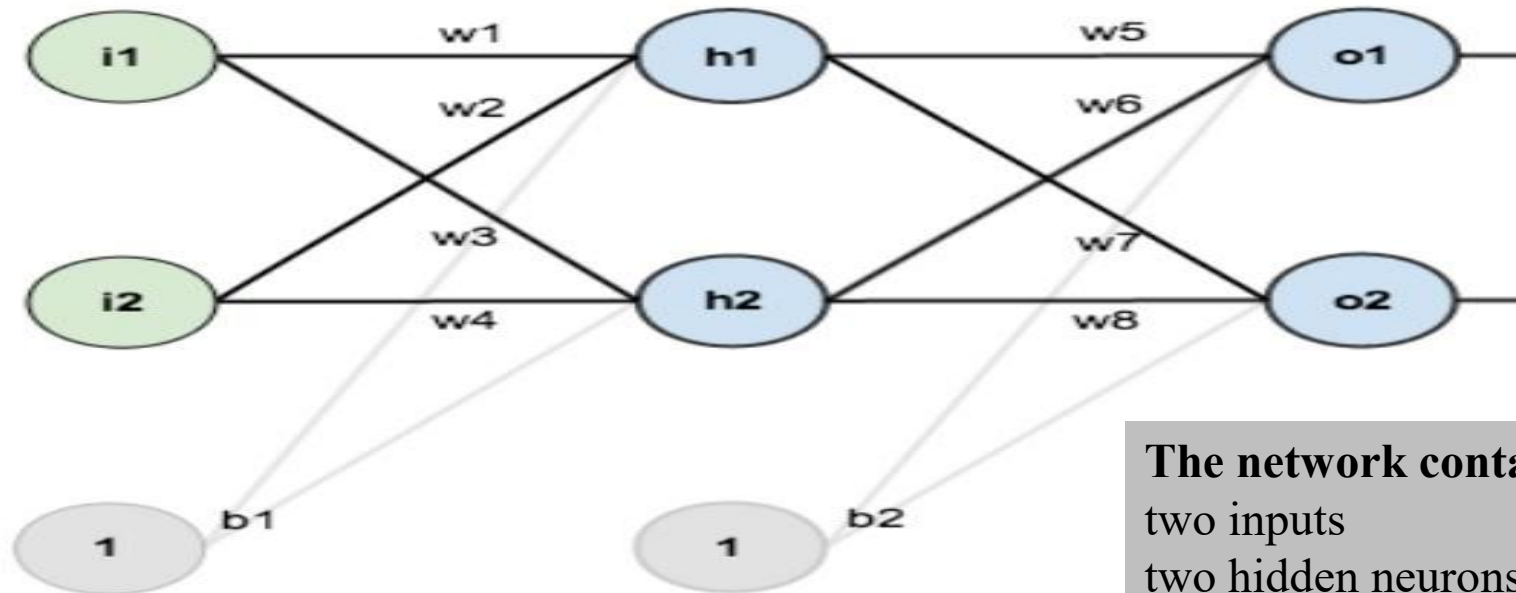
The Backpropagation Algorithm

- ✓ The goal of backpropagation is to optimize the weights so that the neural network can learn how to correctly map arbitrary inputs to outputs.
- ✓ The most basic method of training a neural network is trial and error.
- ✓ If the network isn't behaving the way it should, change the weighting of a random link by a random amount. If the accuracy of the network declines, undo the change and make a different one.
- ✓ It takes time, but the trial and error method does produce results.

Artificial Neural Network

Learning using Back-Propagation

Sample Neural Network to Calculate **Total Error** Using Forward Pass Back propagation



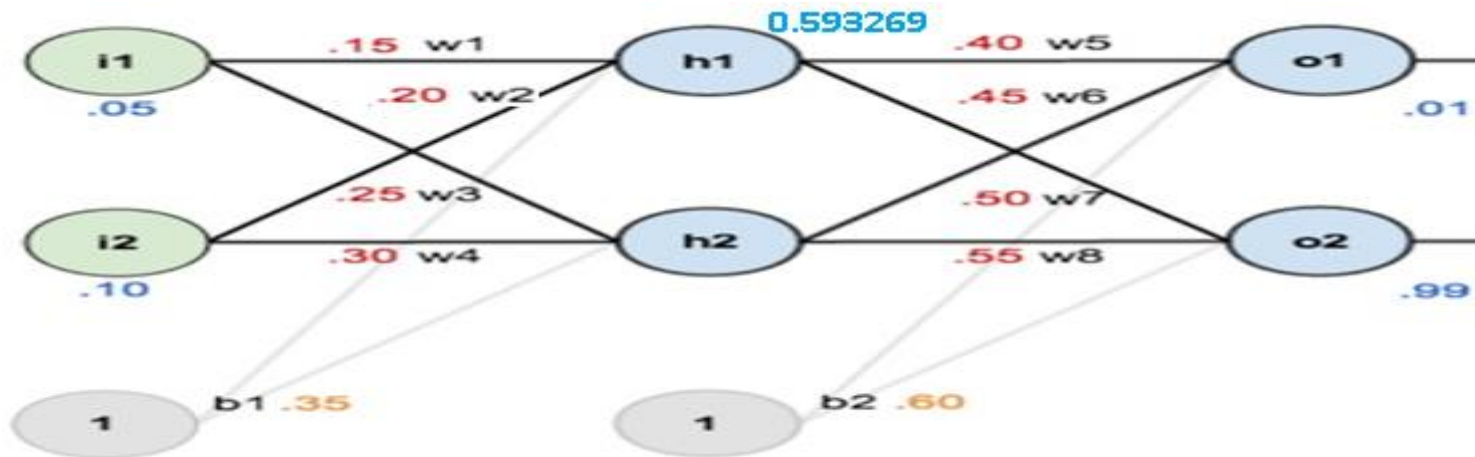
The network contains the following:

- two inputs
- two hidden neurons
- two output neurons
- two biases

Artificial Neural Network

Learning using Back-Propagation: Error Calculation

0.05, 0.10 → 0.01, 0.99



Calculation at node h1

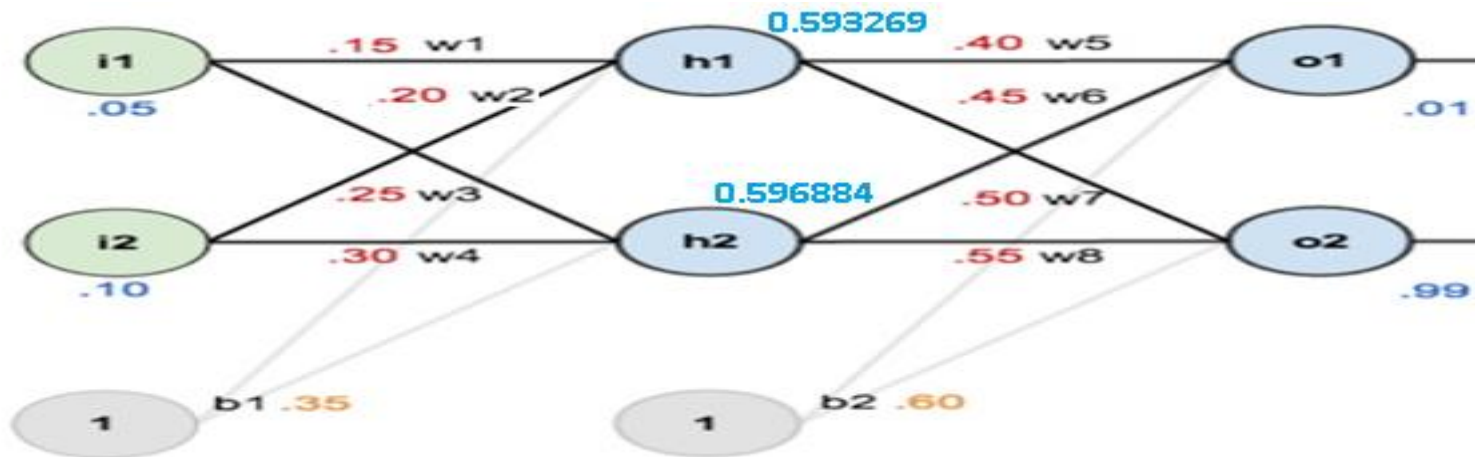
$$net_{h1} = w_1 \times i_1 + w_2 \times i_2 + b_1 \times 1 = 0.15 \times 0.05 + 0.2 \times 0.1 + 0.35 \times 1 = 0.3775$$

$$out_{h1} = \frac{1}{1 + e^{-net_{h1}}} = \frac{1}{1 + e^{-0.3775}} = 0.593269$$

Artificial Neural Network

Learning using Back-Propagation: Error Calculation

0.05, 0.10 → 0.01, 0.99



Calculation at node h_2

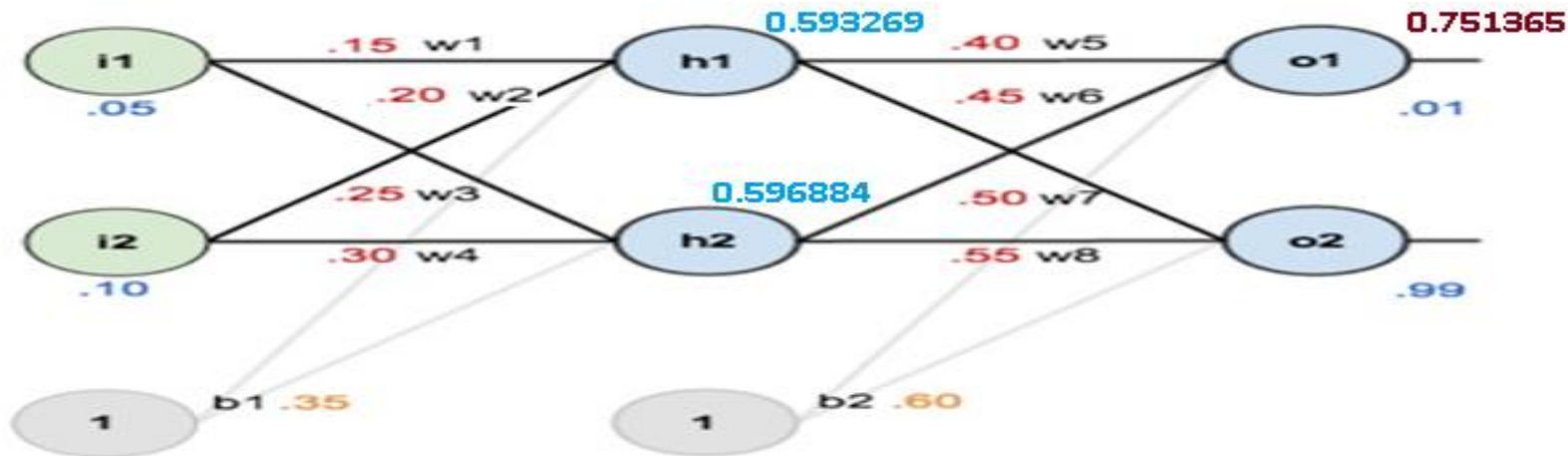
$$net_{h2} = w_3 \times i_1 + w_4 \times i_2 + b_1 \times 1 = 0.25 \times 0.05 + 0.3 \times 0.1 + 0.35 \times 1 = 0.3925$$

$$out_{h2} = \frac{1}{1 + e^{-net_{h1}}} = \frac{1}{1 + e^{-0.3925}} = 0.596884$$

Artificial Neural Network

Learning using Back-Propagation: Error Calculation

0.05, 0.10 → 0.01, 0.99



Calculation at node *o1*

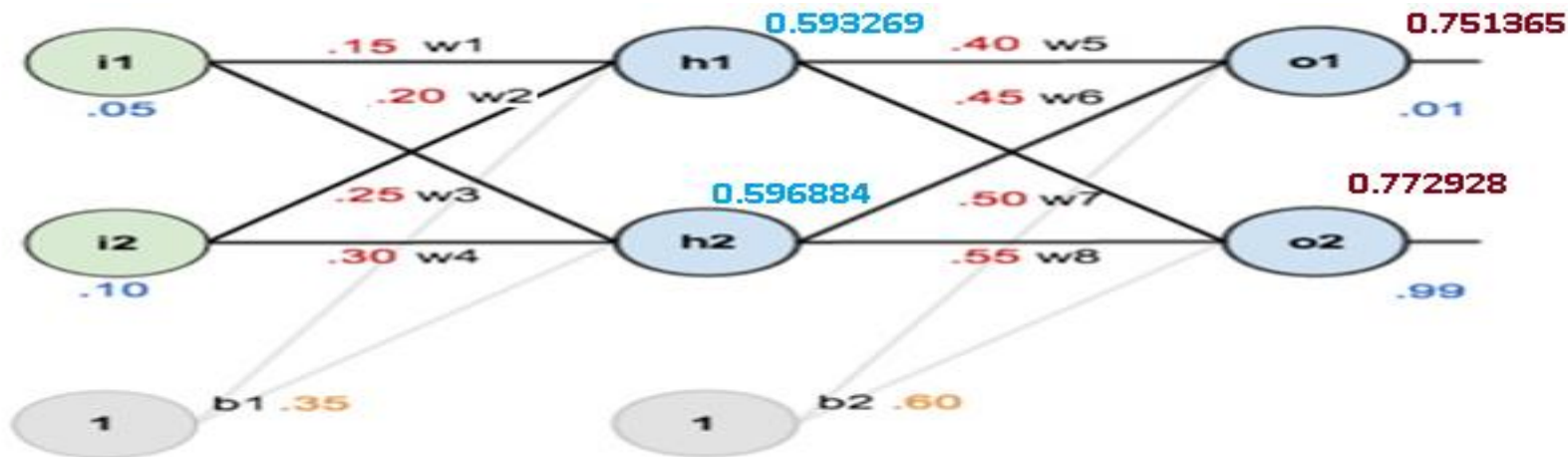
$$net_{o1} = w_5 \times out_{h1} + w_6 \times out_{h2} + b_2 \times 1 = 0.4 \times 0.593269 + 0.45 \times 0.596884 + 0.6 \times 1 = 1.1059054$$

$$out_{o1} = \frac{1}{1 + e^{-net_{o1}}} = \frac{1}{1 + e^{-1.1059054}} = 0.751365$$

Artificial Neural Network

Learning using Back-Propagation: Error Calculation

0.05, 0.10 → 0.01, 0.99



Calculation at node o2

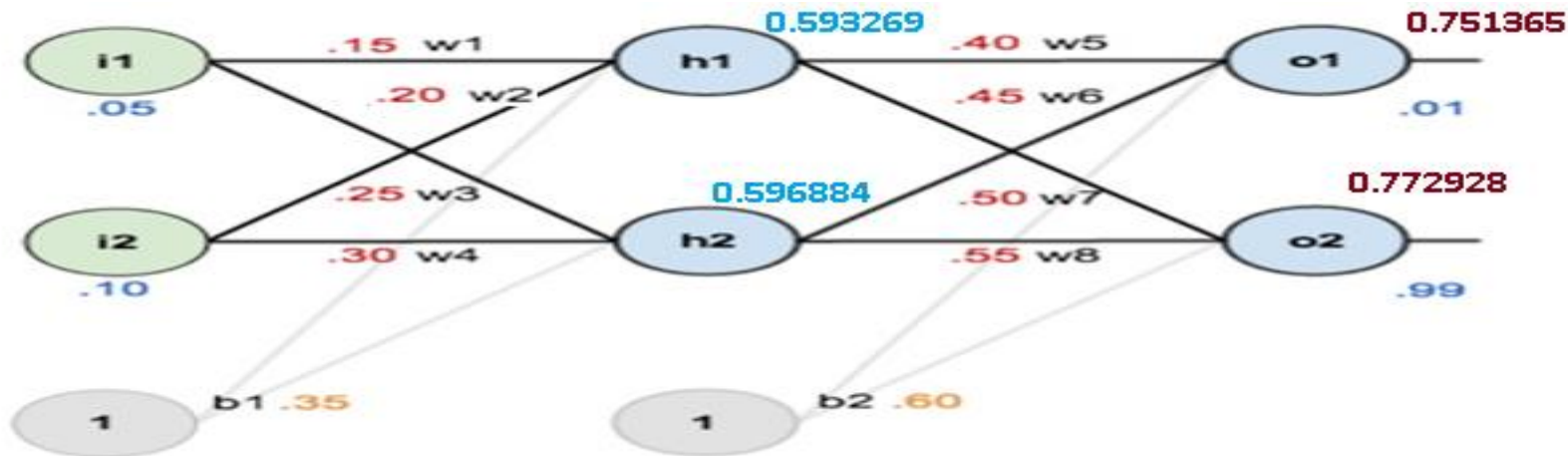
$$net_{o2} = w_7 \times out_{h1} + w_8 \times out_{h2} + b_2 \times 1 = 0.5 \times 0.593269 + 0.55 \times 0.596884 + 0.6 \times 1 = 1.22492$$

$$out_{o2} = \frac{1}{1 + e^{-net_{o2}}} = \frac{1}{1 + e^{-1.22492}} = 0.772928$$

Artificial Neural Network

Propagate the error backward to adjust the weights

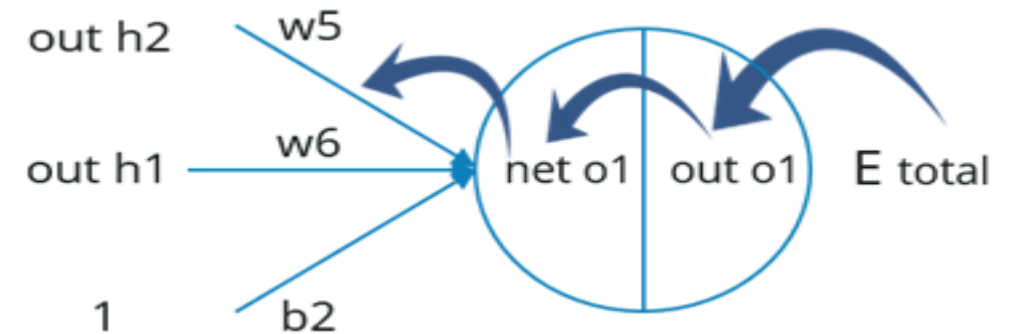
0.05, 0.10 → 0.01, 0.99



Now, we will propagate backwards. This way we will try to reduce the error by changing the values of weights and biases.

Consider **w5**, we will calculate the rate of change of error w.r.t change in weight **w5**.

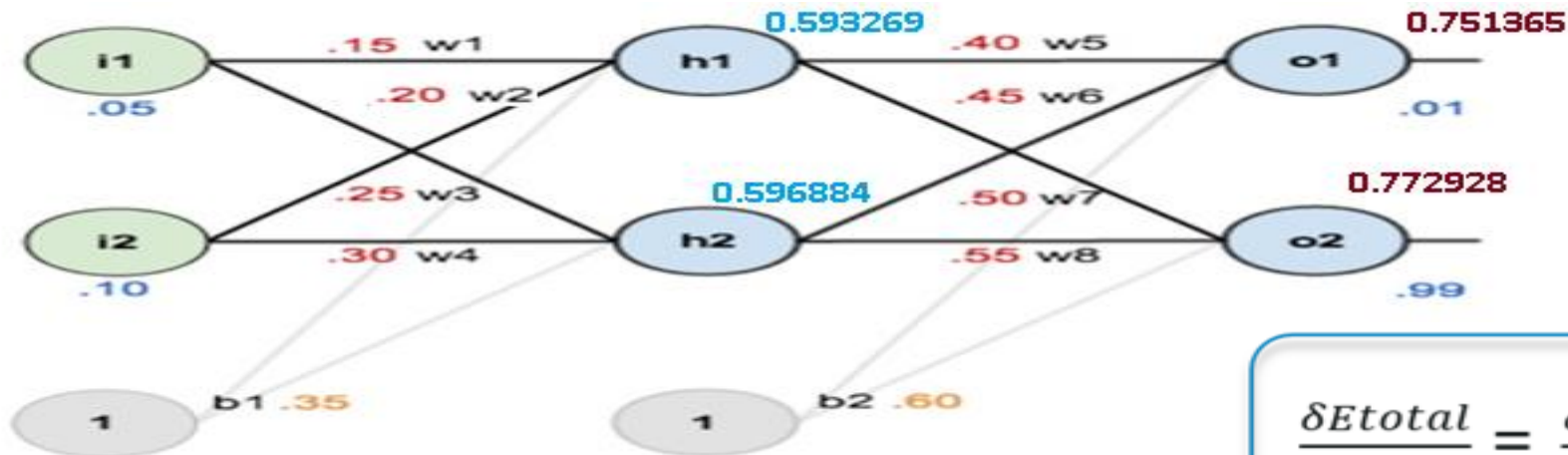
$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$



Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99



Since we are propagating backwards, first thing we need to do is, calculate the change in total errors w.r.t the output **o1** and **o2**.

$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$

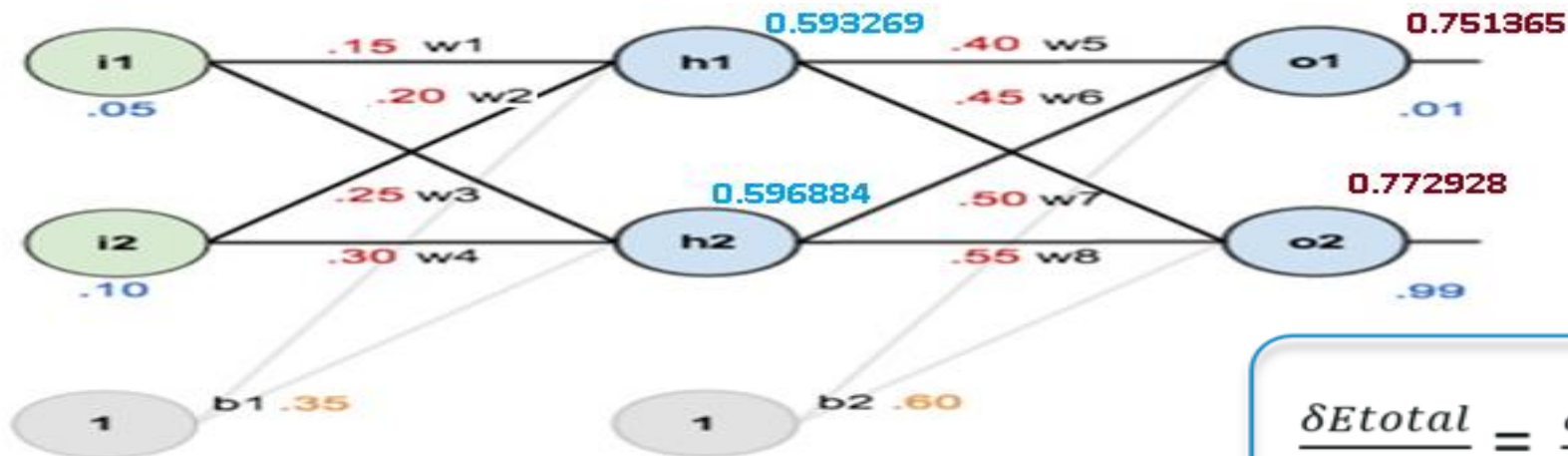
$$E_{total} = 1/2(target\ o1 - out\ o1)^2 + 1/2(target\ o2 - out\ o2)^2$$

$$\frac{\delta E_{total}}{\delta out\ o1} = -(target\ o1 - out\ o1) = -(0.01 - 0.75136507) = 0.74136507$$

Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99



Now, we will propagate further backwards and calculate the change in output **o1** w.r.t to its total net input.

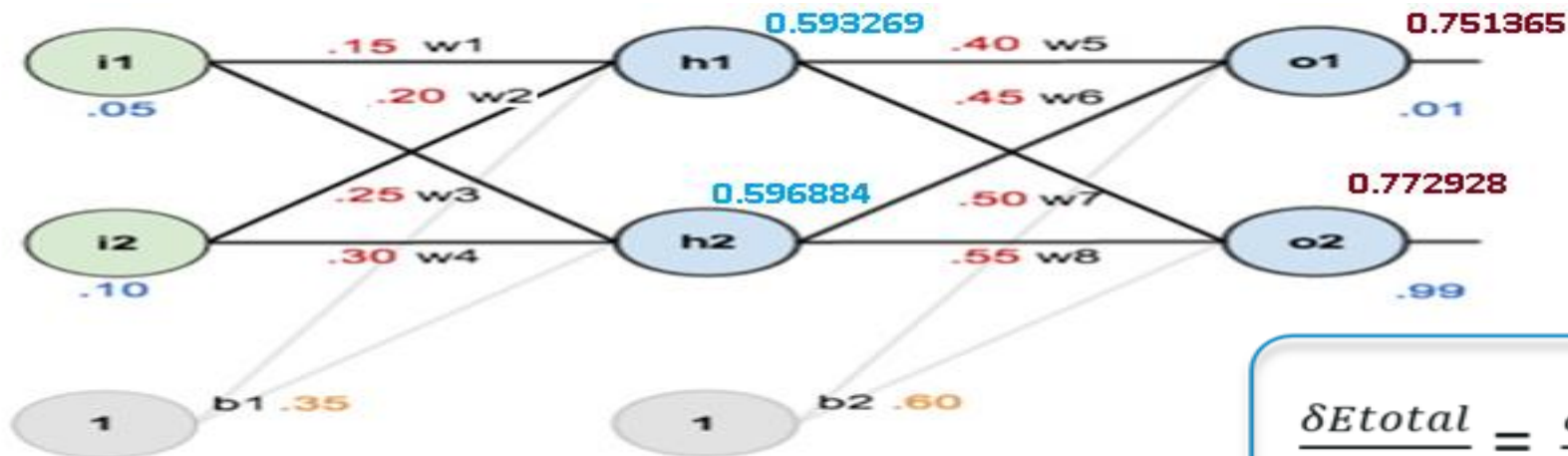
$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$

$$\frac{\delta out\ o1}{\delta net\ o1} = out\ o1 (1 - out\ o1) = 0.75136507 (1 - 0.75136507) = 0.186815602$$

Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99



Let's see now how much does the total net input of o1 changes w.r.t w5?

$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$

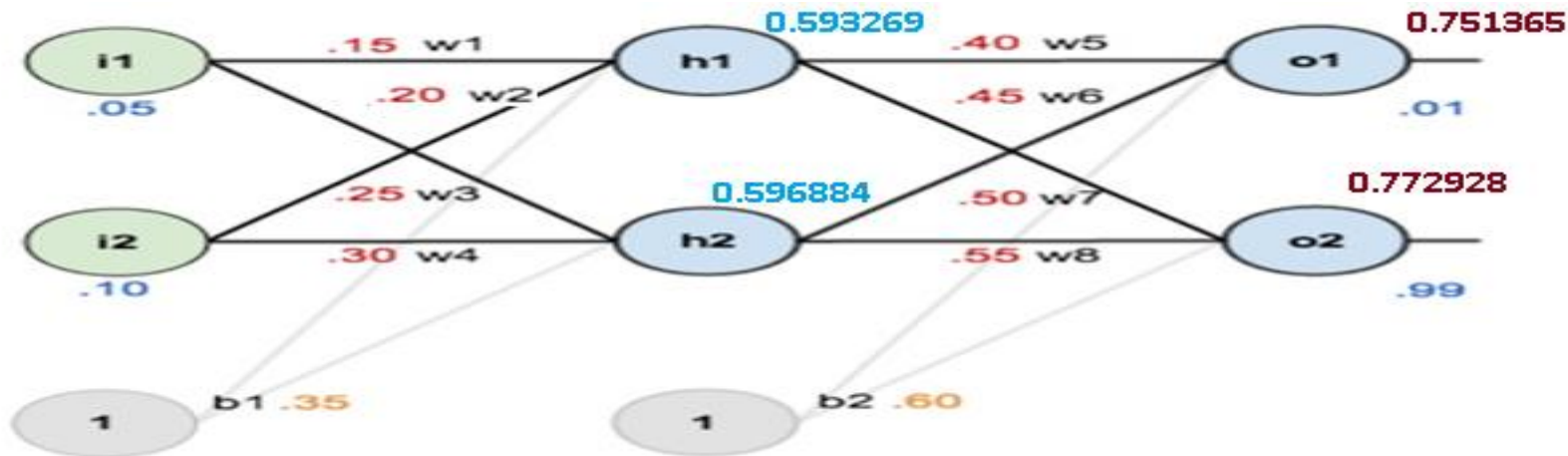
$$net\ o1 = w_5 * out\ h1 + w_6 * out\ h2 + b_2 * 1$$

$$\frac{\delta net\ o1}{\delta w_5} = 1 * out\ h1 * w_5^{(1-1)} + 0 + 0 = 0.5932699992$$

Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99



Putting all the values together and calculating the updated weight value

$$\frac{\delta E_{total}}{\delta w_5} = \frac{\delta E_{total}}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w_5}$$

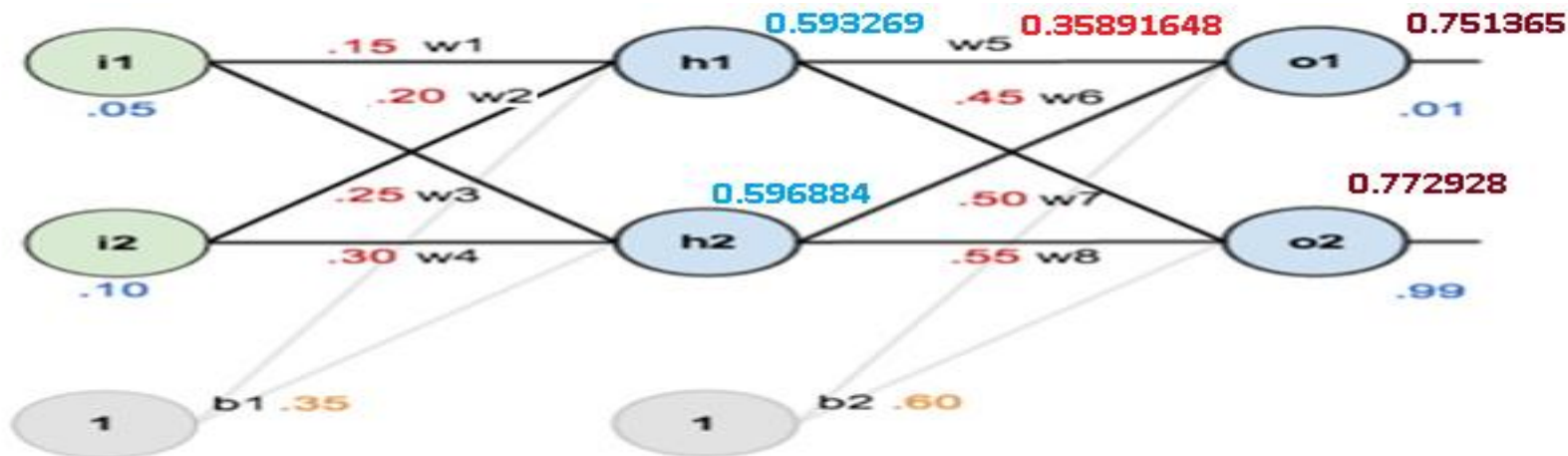
0.082167041

Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99

Let's calculate the updated value of w5:



$$w5^+ = w5 - n \frac{\delta E_{total}}{\delta w5}$$

$$w5^+ = 0.4 - 0.5 * 0.082167041$$

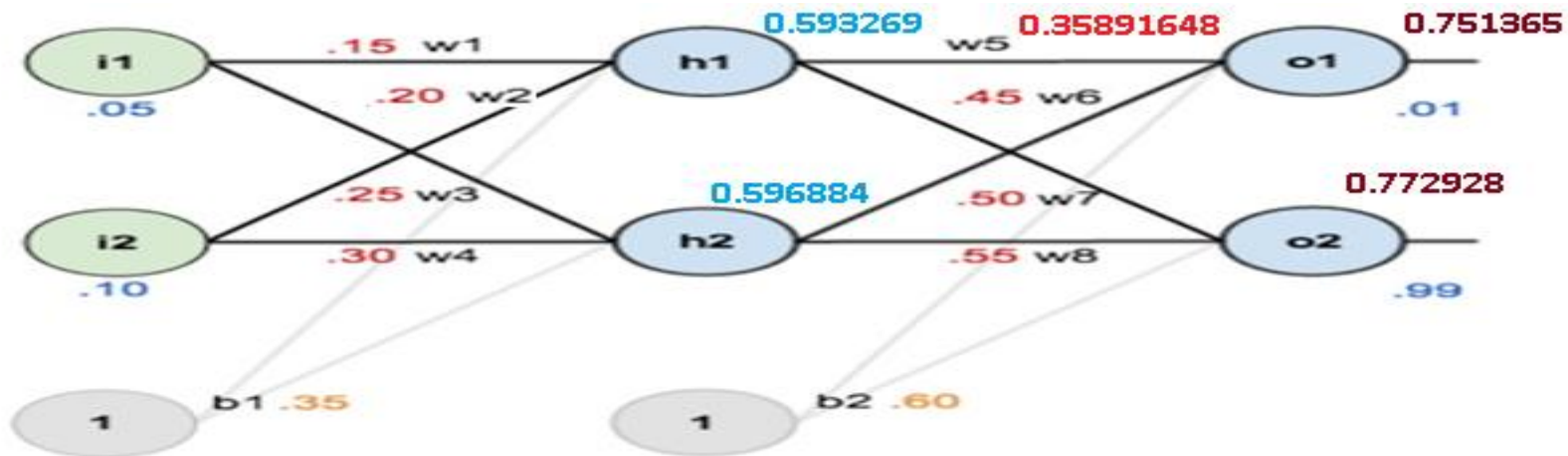
Updated w5

0.35891648

Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99



We can repeat this process to get the new weights w_6 , w_7 , and w_8 :

$$w_6^+ = 0.408666186$$

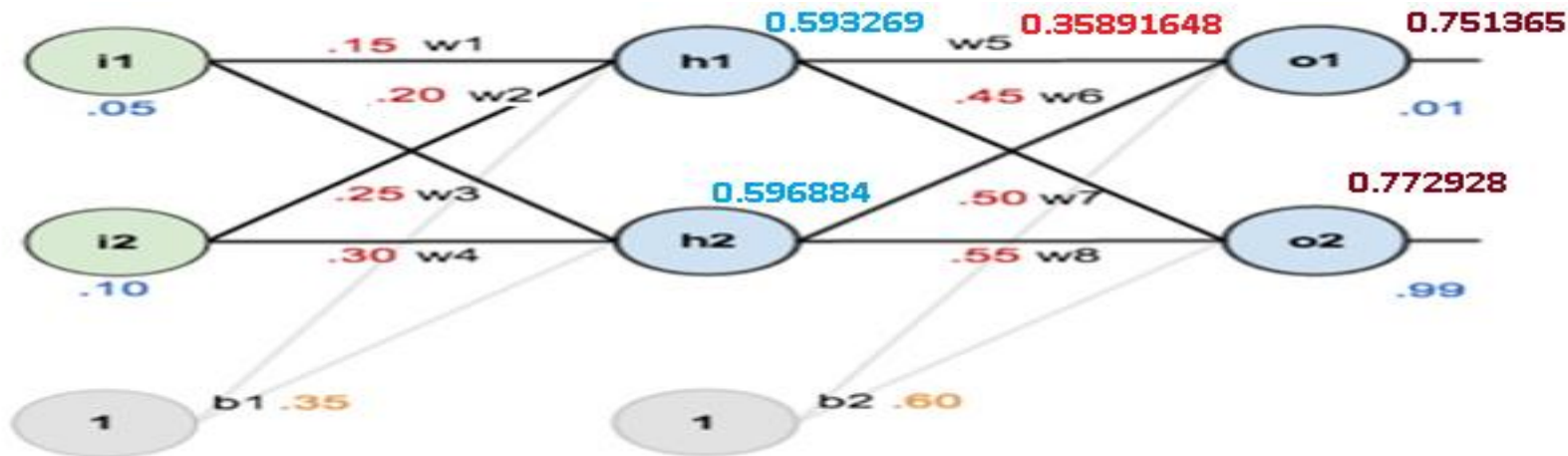
$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

Artificial Neural Network

Propagate the error backward to adjust the weights

0.05, 0.10 → 0.01, 0.99

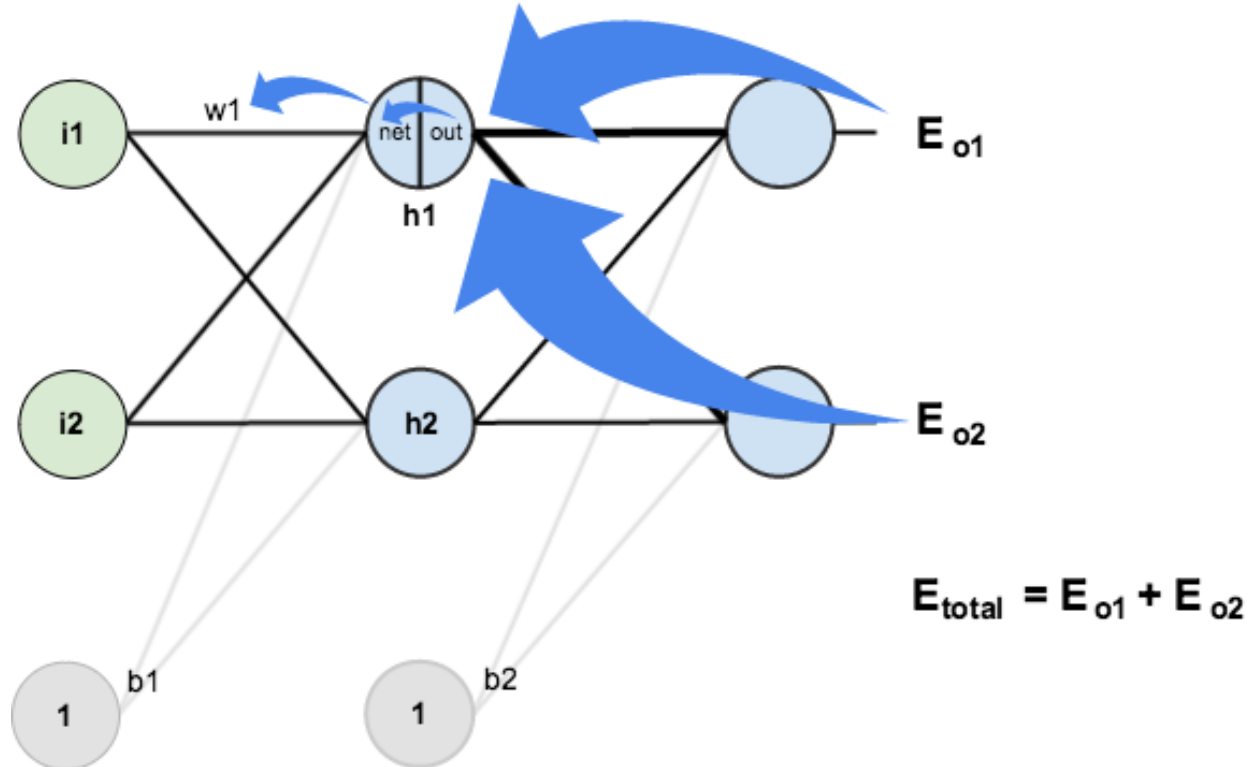


- Similarly, we can calculate the other weight values as well.
- After that we will again propagate forward and calculate the output. Again, we will calculate the error.
- If the error is minimum we will stop right there, else we will again propagate backwards and update the weight values.
- This process will keep on repeating until error becomes minimum.

Learning using Back Propagation

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$
$$\downarrow$$
$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$

Next, we'll continue the backwards pass by calculating new values for w_1 , w_2 , w_3 and w_4



Learning using Back Propagation

Starting with $\frac{\partial E_{o1}}{\partial out_{h1}}$:

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}}$$

We can calculate $\frac{\partial E_{o1}}{\partial net_{o1}}$ using values we calculated earlier:

$$\frac{\partial E_{o1}}{\partial net_{o1}} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} = 0.74136507 * 0.186815602 = 0.138498562$$

And $\frac{\partial net_{o1}}{\partial out_{h1}}$ is equal to w_5 :

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial out_{h1}} = w_5 = 0.40$$

Learning using Back Propagation

Plugging them in:

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} = 0.138498562 * 0.40 = 0.055399425$$

Following the same process for $\frac{\partial E_{o2}}{\partial out_{h1}}$, we get:

$$\frac{\partial E_{o2}}{\partial out_{h1}} = -0.019049119$$

Therefore:

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} = 0.055399425 + -0.019049119 = 0.036350306$$

Learning using Back Propagation

Now that we have $\frac{\partial E_{total}}{\partial out_{h1}}$, we need to figure out $\frac{\partial out_{h1}}{\partial net_{h1}}$ and then $\frac{\partial net_{h1}}{\partial w}$ for each weight:

$$out_{h1} = \frac{1}{1 + e^{-net_{h1}}}$$

$$\frac{\partial out_{h1}}{\partial net_{h1}} = out_{h1}(1 - out_{h1}) = 0.59326999(1 - 0.59326999) = 0.241300709$$

We calculate the partial derivative of the total net input to h_1 with respect to w_1 the same as we did for the output neuron:

$$net_{h1} = w_1 * i_1 + w_3 * i_2 + b_1 * 1$$

$$\frac{\partial net_{h1}}{\partial w_1} = i_1 = 0.05$$

Learning using Back Propagation

Putting it all together:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 * 0.241300709 * 0.05 = 0.000438568$$

We can now update w_1 :

$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

Learning using Back Propagation

Repeating this for w_2 , w_3 , and w_4

$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

Finally, we've updated all of our weights! When we fed forward the 0.05 and 0.1 inputs originally, the error on the network was 0.298371109. After this first round of backpropagation, the total error is now down to 0.291027924.

It might not seem like much, but after repeating this process 10,000 times, for example, the error plummets to 0.0000351085. At this point, when we feed forward 0.05 and 0.1, the two outputs neurons generate 0.015912196 (vs 0.01 target) and 0.984065734 (vs 0.99 target).

Learning using Back Propagation

Similarly, we can calculate the other weight values as well.

After that we will again propagate forward and calculate the output. Again, we will calculate the error.

If the error is minimum we will stop right there, else we will again propagate backwards and update the weight values.

This process will keep on repeating until error becomes minimum.